

Detección de Objetos

De CNN a transformers: bounding boxes, regiones y arquitecturas modernas

Sergio Hernández, PhD
shernandez@ucm.cl



Magíster en Data Science — Universidad Católica del Maule



Clasificación de Imágenes

- ▶ Pregunta: ¿Qué objeto está en la imagen?
- ▶ Respuesta: Una etiqueta de clase.
- ▶ Salida: Probabilidades por clase.

Detección de Objetos

- ▶ Pregunta: ¿Dónde y qué objetos están?
- ▶ Respuesta: Ubicación + clase de cada objeto.
- ▶ Salida: Múltiples bounding boxes + etiquetas.

Dominio de Aplicación Real

- ▶ **Vehículos autónomos:** Detectar peatones, vehículos, señales de tráfico.
- ▶ **Vigilancia:** Identificación de amenazas, conteo de personas.
- ▶ **Medicina:** Localización de tumores, anomalías en radiografías.
- ▶ **Manufactura:** Inspección de calidad, detección de defectos.
- ▶ **Análisis de video:** Seguimiento de objetos en escenas complejas.

Desafío

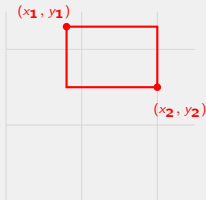
Localizar y clasificar múltiples objetos simultáneamente, a diferentes escalas y con posibles oclusiones.

Bounding Boxes: Representación Fundamental

Un **bounding box** es un rectángulo que delimita un objeto detectado.

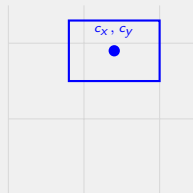
Esquinas (x_1, y_1, x_2, y_2)

Coordenadas de esquina superior-izquierda e inferior-derecha.



Centro (c_x, c_y, w, h)

Centro, ancho y alto.



Conversión entre Representaciones

Las dos representaciones son equivalentes.

$$c_x = \frac{x_1 + x_2}{2}, \quad c_y = \frac{y_1 + y_2}{2}, \quad w = x_2 - x_1, \quad h = y_2 - y_1$$

$$x_1 = c_x - \frac{w}{2}, \quad y_1 = c_y - \frac{h}{2}, \quad x_2 = c_x + \frac{w}{2}, \quad y_2 = c_y + \frac{h}{2}$$

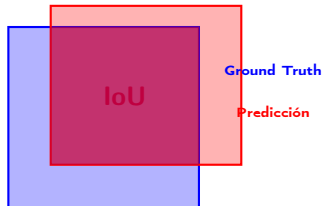
Almacenamiento

Los bounding boxes se representan como tensores de forma $(n, 4)$, donde n es el número de cajas.

Intersección sobre Unión (IoU)

La métrica **IoU** cuantifica qué tan bien coinciden las predicciones con la verdad.

$$\text{IoU} = \frac{\text{Intersección}}{\text{Unión}}$$



- ▶ $\text{IoU} \geq 0.5$: Positivo verdadero (estándar)
- ▶ $\text{IoU} = 1.0$: Predicción perfecta

Basados en Regiones

- ▶ Proponen regiones candidatas.
- ▶ Clasifican y refinan.
- ▶ Ejemplo: Faster R-CNN
- ▶ Lentos pero precisos.

Una Etapa

- ▶ Predicción directa.
- ▶ Ejemplo: YOLO, SSD
- ▶ Rápidos pero menos precisos.
- ▶ Tiempo real.

Transformers

- ▶ Contexto global.
- ▶ Ejemplo: DETR
- ▶ Rendimiento equilibrado.
- ▶ Arquitectura limpia.

R-CNN (Girshick et al., 2014) fue el primer método exitoso con redes profundas.

Pipeline de 4 Pasos

1. **Propostas:** Generar ~2000 regiones candidatas (búsqueda selectiva).
2. **CNN:** Extraer características de cada región con AlexNet.
3. **Clasificación:** Clasificar con SVM.
4. **Regresión:** Ajustar coordenadas del bounding box.

Limitación

Muy lento: ~50 segundos por imagen (necesita 2000 pasadas).

Arquitectura R-CNN: AlexNet de 8 Capas

R-CNN utiliza AlexNet como red convolucional base.

Backbone CNN

- ▶ **AlexNet:** 8 capas (5 conv + 3 FC)
- ▶ **Conv layers 1-5:**
 - ▶ L1: 96 filtros, 11×11 , stride 4, padding 0
 - ▶ L2: 256 filtros, 5×5 , stride 1, padding 2
 - ▶ L3-5: 384/384/256, 3×3 , stride 1, padding 1
- ▶ Max pooling: 3×3 , stride 2
- ▶ Entrada: 227×227 RGB
- ▶ Salida: $6 \times 6 \times 256$ (después últimas conv)

Head (Clasificación + Regresión)

- ▶ **FC6-FC8:** 3 capas densas
 - ▶ FC6: 4096 unidades
 - ▶ FC7: 4096 unidades
 - ▶ FC8: 21 unidades (20 clases + background)
- ▶ ReLU activación entre capas
- ▶ Dropout: 0.5
- ▶ **Regresión:** 4 salidas ($\Delta x, \Delta y, \Delta w, \Delta h$)

Fast R-CNN: Arquitectura con RoI Pooling

Las mejoras posteriores aceleraron drásticamente el método original.

Backbone: ResNet-50 o ResNet-101

- ▶ **ResNet-50:** 50 capas en 4 bloques residuales
 - ▶ Conv1: 7×7 , stride 2, padding 3 (input 224×224)
 - ▶ Residual blocks: layer1-4 con stride 2 entre bloques
 - ▶ Bottleneck bloques: $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$ convolutions
 - ▶ Output stride 32 ($H/W \div 32$)

RoI Pooling

- ▶ Aplica max pooling a cada región.
- ▶ Input: Feature map + coordenadas de RoI
- ▶ Output: $7 \times 7 \times 512$ (uniforme)
- ▶ Conexiones posteriores al head

Head Dense

- ▶ FC6: 4096 unidades, ReLU
- ▶ FC7: 4096 unidades, ReLU
- ▶ FC8-clas: #clases salidas
- ▶ FC8-reg: 4 salidas ($\Delta x, \Delta y, \Delta w, \Delta h$)
- ▶ Dropout: 0.5

Faster R-CNN: Region Proposal Network

Faster R-CNN añade RPN para propuestas automáticas.

Backbone: ResNet-50/101 (igual que Fast R-CNN)

Entrada: imagen 1000×1000 , Output: feature map $H/32 \times W/32$

Region Proposal Network

- ▶ **3×3 conv:** 512 filtros, stride 1, padding 1
- ▶ **Rama cls:** 1×1 conv, 2 outputs (object/background)
- ▶ **Rama reg:** 1×1 conv, 4 outputs ($\Delta x, \Delta y, \Delta w, \Delta h$)
- ▶ **Anchors:** 9 anchors por píxel (3 escalas $\times$ 3 ratios)
- ▶ **NMS:** Umbral IoU 0.7

Head Detection

- ▶ RoI Align (mejora de RoI Pooling)
- ▶ Output: $7 \times 7 \times 512$
- ▶ FC6-FC7: 1024 unidades cada una
- ▶ FC-cls: #clases salidas
- ▶ FC-reg: $4 \times$ #clases salidas
- ▶ Pérdida multi-tarea: $L_{cls} + \lambda L_{reg}$

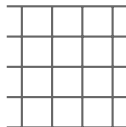
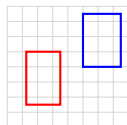
YOLO: You Only Look Once

YOLO (Redmon et al., 2016) cambio el paradigma: **una sola pasada**.

Idea: Regresión Directa

Divide la imagen en cuadrícula $S \times S$ y predice:

- ▶ Confianza de objeto en cada celda.
- ▶ Clase del objeto.
- ▶ Desplazamiento del bounding box.



Predicciones

Cuadrícula

- ▶ Ventaja: 45 FPS (tiempo real)
- ▶ Desventaja: Menos preciso en objetos pequeños

YOLOv1: Arquitectura Completa de 24 Capas

YOLOv1 utiliza una red simple pero efectiva de 24 capas convolucionales.

Backbone Convolucional

- ▶ **7 conv layers:** 64-128-256-512 filtros
 - ▶ Conv1: 7×7 , stride 2, padding 3 (input 448×448)
 - ▶ Conv2: 3×3 , stride 2, padding 1
 - ▶ Intercaladas con max pooling 2×2 , stride 2
- ▶ Output: $7 \times 7 \times 512$ (grid $S=7$)
- ▶ Activación: ReLU (excepto última capa sigmoid)

Head Dense (Predicciones)

- ▶ FC1: 4096 unidades, ReLU
- ▶ FC2: $7 \times 7 \times 30 = 1470$ unidades
 - ▶ 7×7 celdas
 - ▶ 2 bounding boxes por celda ($4 + 1$ confianza = 5)
 - ▶ 20 clases probabilidades
 - ▶ Total: $7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$

YOLOv5: Arquitectura Moderna con CSPDarknet

YOLOv5 introduce mejoras arquitecturales y de entrenamiento significativas.

Backbone: CSPDarknet

- ▶ **CSP:** Cross Stage Partial
- ▶ 5 bloques CSP con estructura residual
- ▶ Stride 2 entre bloques
- ▶ Output 3 feature maps:
 - ▶ P3: $H/8 \times W/8$ (pequeños)
 - ▶ P4: $H/16 \times W/16$ (medianos)
 - ▶ P5: $H/32 \times W/32$ (grandes)
- ▶ Activación: SiLU (Swish)

Neck: FPN

- ▶ **Feature Pyramid Network**
- ▶ Conexiones up-sampling + down-sampling
- ▶ Fusión multi-escala de características
- ▶ Concatenación + 1×1 convs

Head: Predicciones Multi-Escala

3 cabezas de detección (una por escala): Conv 1×1 por escala, output (x,y,w,h,conf,cls1,...,cls80)

SSD300: Arquitectura Multi-Escala con VGG-16

SSD300 usa VGG-16 truncado como backbone base.

Backbone: VGG-16 Truncado

- ▶ VGG-16: 16 capas conv
- ▶ Primeras 13 conv layers
 - ▶ Bloques de 2-3 convs 3×3
 - ▶ Stride 1, max pool stride 2
 - ▶ Canales: 64-128-256-512
- ▶ **Remover:** Últimas FC layers
- ▶ Input: 300×300 RGB
- ▶ Output: $38\times38\times512$

Capas Adicion. Multi-Escala

- ▶ Conv 1×1 : 1024 filtros
- ▶ Conv 1×1 : 1024 filtros
- ▶ Series de conv 3×3 stride 2
 - ▶ Stride 2: 512
 - ▶ Stride 2: 256
 - ▶ Stride 2: 256
 - ▶ Stride 2: 256
- ▶ Feature maps:
 - ▶ 38×38 (VGG)
 - ▶ 19×19 , 10×10 , 5×5 , 3×3 , 1×1

SSD: Heads de Detección Multi-Escala

Para cada feature map, se aplican convoluciones 3×3 pequeñas.

Predicción por Escala

Para cada feature map de tamaño $m \times m$:

- ▶ **Conv 3×3 :** Aplicada a cada posición spatial
- ▶ Para cada anchor: predice $(x, y, w, h, c_0, \dots, c_{19})$
- ▶ Múltiples anchors por posición (4-6 formas y escalas)
- ▶ Salida: $m \times m \times (\#anchors \times (4 + \#clases))$

Ejemplo SSD300

- ▶ **38×38 :** 4 anchors $\rightarrow 38 \times 38 \times 96$ (4 coords + 20 clases) $\times 4$
- ▶ **19×19 :** 6 anchors $\rightarrow 19 \times 19 \times 144$
- ▶ **10×10 :** 6 anchors $\rightarrow 10 \times 10 \times 144$
- ▶ **5×5 :** 6 anchors $\rightarrow 5 \times 5 \times 144$
- ▶ **3×3 :** 4 anchors $\rightarrow 3 \times 3 \times 96$
- ▶ **1×1 :** 4 anchors $\rightarrow 1 \times 1 \times 96$
- ▶ **Total:** ~8732 predicciones de bounding box

DETR: Detection Transformer - Cambio de Paradigma

DETR (Carion et al., 2020) reformula detección como **predicción directa de conjuntos**.

Cambio de Paradigma

- ▶ **Enfoque tradicional:** Genera propuestas de regiones, filtra, refina (multi-etapa).
- ▶ **DETR:** Predice directamente un conjunto fijo de detecciones en paralelo.
- ▶ Trata detección como un problema de asignación de conjuntos.
- ▶ Elimina componentes manuales como NMS, anchor boxes.

Arquitectura

- ▶ **Encoder CNN:** Extrae características espaciales.
- ▶ **Transformer:** Captura relaciones globales.
- ▶ **Decoder:** Predice clase y bbox para N objetos.

DETR predice un conjunto de N detecciones (donde $N \gg$ número real de objetos).

Formulación del Problema

- ▶ Predice: $\hat{\mathcal{Y}} = \{(\hat{c}_i, \hat{b}_i)\}_{i=1}^N$ donde $\hat{c}_i$ es clase y $\hat{b}_i$ es bounding box.
- ▶ N es fijo (ej. $N=100$) $\gg$ número real de objetos.
- ▶ Muchas predicciones serán clase “vacío” (no-object).
- ▶ No hay orden: dos permutaciones del conjunto tienen el mismo significado.

Ventaja Clave

Al ser un conjunto, el orden no importa. Esto permite decodificación **paralela** en lugar de secuencial.

Emparejamiento Bipartito y el Algoritmo Húngaro

El desafío: asignar predicciones a objetos verdaderos de forma óptima.

Problema de Asignación

- ▶ Tenemos N predicciones y (típicamente) mucho menos objetos verdaderos.
- ▶ Necesitamos encontrar permutación óptima $\sigma \in \mathcal{S}_N$.
- ▶ Objetivo: minimizar costo de emparejamiento total.

$$\hat{\sigma} = \arg \min_{\sigma} \sum_{i=1}^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)})$$

Solución: Algoritmo Húngaro

- ▶ Resuelve el problema de asignación óptima en tiempo polinomial.
- ▶ Cada predicción se empareja con un objeto verdadero (o vacío).
- ▶ Evita duplicados: una predicción = un objeto o vacío.
- ▶ Eficiencia: $O(n^3)$ es manejable para $N=100$.

Costo de Emparejamiento: Combinación de Clase y Localización

El costo de emparejamiento combina información de clasificación y bbox.

$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) = -\mathbb{1}_{c_i \neq \emptyset} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{c_i \neq \emptyset} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

Componentes

- ▶ **Término de clasificación:** $-\hat{p}_{\sigma(i)}(c_i)$ = probabilidad negativa de clase correcta.
- ▶ **Término de caja:** Solo se aplica si $c_i \neq \emptyset$ (hay un objeto real).
- ▶ Los indicadores $\mathbb{1}_{c_i \neq \emptyset}$ aseguran que no se penaliza perder “no-objetos”.

Característica Clave

Diferente a métodos tradicionales: no usa NMS post-procesamiento, sino matching óptimo durante entrenamiento.

Pérdida Húngara: Entrenamiento del Modelo

Después del emparejamiento óptimo, se calcula la pérdida total para backpropagation.

$$\mathcal{L}_{\text{Hungarian}} = \sum_{i=1}^N \left[-\log \hat{p}_{\hat{\sigma}(i)}(c_i) + \mathbb{1}_{c_i \neq \emptyset} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\hat{\sigma}(i)}) \right]$$

Componentes

- ▶ **Pérdida de clasificación:** Entropía cruzada en probabilidades de clase predichas.
- ▶ **Pérdida de bbox:** Solo se aplica a objetos reales (no a predicciones vacías).
- ▶ **Ponderación:** Se reduce el peso de “no-objeto” por factor $\lambda = 10$ para evitar dominancia (99% de predicciones serán vacías).

Manejo del Desbalance de Clases

La mayoría de predicciones son vacías, por eso se pondera: $\mathcal{L}_{\emptyset} = 0.1 \times \mathcal{L}_{\text{matched}}$.

Pérdida de Caja: Combinación de GloU y L1

Para localización de bounding boxes, se combina dos pérdidas complementarias.

$$\mathcal{L}_{\text{box}} = \lambda_{\text{iou}} \mathcal{L}_{\text{GloU}} + \lambda_{\text{L1}} \left\| b_i - \hat{b}_{\hat{\sigma}(i)} \right\|_1$$

Pérdida GloU

- ▶ *Generalized IoU*: resuelve problema de IoU cuando boxes no se superponen.
- ▶ **Escala invariante**: penalización homogénea para cajas grandes y pequeñas.
- ▶ Rango: $[-1, 1]$ (-1 = cajas opuestas, 1 = perfectas).

Pérdida L1

- ▶ Captura precisión en coordenadas (píxeles).
- ▶ **Sensible a escala**: mejor para ajustes finos.
- ▶ Complementa GloU: GloU para forma, L1 para precisión.

Ponderación

Típicamente: $\lambda_{\text{iou}} = 2$, $\lambda_{\text{L1}} = 5$ para balancear contribuciones.

DETR: Arquitectura Completa

DETR combina backbone CNN con transformers para predicción de conjuntos.

Backbone CNN

- ▶ ResNet-50 o ResNet-101
- ▶ 50/101 capas conv
- ▶ Stride 32
- ▶ Input: 800×1066 (~ 3 MP)
- ▶ Output: $25 \times 34 \times 2048$ features

Encoder Transformer

- ▶ 6 capas de transformer
- ▶ Multi-head attention: 8 heads
- ▶ Hidden: 256 dims
- ▶ FFN: 2048 dims
- ▶ Positional encoding: sinusoidal
- ▶ Dropout: 0.1

Decoder Transformer

- ▶ 6 capas de transformer
- ▶ Multi-head attention: 8 heads
- ▶ Hidden: 256 dims
- ▶ FFN: 2048 dims
- ▶ *Learned queries*: $N=100$
- ▶ Self-attention + cross-attention

DETR: Heads de Clasificación y Regresión

Después del decoder, heads específicos predicen clases y bboxes.

Head de Clasificación

- ▶ Input: Decoder output, 256D por query
- ▶ 3 FC layers:
 - ▶ FC1: 256D
 - ▶ FC2: 256D
 - ▶ FC3: $(\text{num_classes} + 1)\text{D}$
- ▶ +1 para clase "sin objeto"
- ▶ Softmax: probabilidades
- ▶ Comparte pesos entre queries

Head de Regresión

- ▶ Input: Decoder output, 256D
- ▶ 3 FC layers:
 - ▶ FC1: 256D
 - ▶ FC2: 256D
 - ▶ FC3: 4D (x, y, w, h)
- ▶ Regresión directa (sin sigmoid)
- ▶ Pesos compartidos entre queries
- ▶ Normalización: coords relativo a imagen

Simplicidad Conceptual

- ▶ Un modelo unificado: no necesita componentes separados (RPN, NMS, etc.).
- ▶ Arquitectura extremo-a-extremo entrenable.
- ▶ Formulación matemática clara y elegante.

Ventajas Prácticas

- ▶ **Sin NMS:** El matching óptimo reemplaza el post-procesamiento manual.
- ▶ **Decodificación paralela:** Todos los N objetos se predicen simultáneamente.
- ▶ **Menos hipótesis manuales:** No requiere diseño de anchors, ratios, escalas.
- ▶ **Generalización:** Fácilmente extensible a panóptica, instancia, etc.

YOLOv5 y Evolución Reciente

La evolución de YOLO demuestra la importancia de la **optimización arquitectural y de entrenamiento**.

Mejoras en YOLOv5

- ▶ **Backbone optimizado:** CSPDarknet con conexiones residuales mejoradas.
- ▶ **Feature Pyramid Network (FPN):** Extracción multi-escala eficiente.
- ▶ **Activaciones mejoradas:** Reemplazo de ReLU con activaciones más sofisticadas.
- ▶ **Data augmentation agresiva:** Mosaic augmentation, mixup.
- ▶ Logra 50.2 mAP@0.5:0.95 en COCO (competitivo con Faster R-CNN).

Trend Actual: Eficiencia

- ▶ Métodos ligeros para dispositivos móviles y edge.
- ▶ Compresión de modelos: pruning, quantización, destilación.
- ▶ Trade-off precisión-velocidad mejor investigado.

- ▶ **Detección de objetos** es fundamental en visión computacional y aplicaciones del mundo real.
- ▶ Tres paradigmas con trade-offs claros:
 - ▶ Basados en regiones → máxima precisión.
 - ▶ Una etapa → máxima velocidad.
 - ▶ Transformers → arquitectura más limpia.
- ▶ **Métricas:** IoU (localización), mAP (desempeño), FPS (velocidad).
- ▶ **Fronteras:** Fusión multi-modal, detección 3D, video, objetos pequeños.
- ▶ Elección de método según dominio y restricciones de recursos.